

PERSONAL DIGITAL VAULT

GitHub Clone & Run Guide

Submission-ready instructions for cloning the project on a fresh computer and running it successfully.

Project	Personal Digital Vault / VaultDesk
Source	GitHub repository
Platform	Windows + .NET + Microsoft SQL Server
Purpose	Project cloning and technical submission

Important: The exact GitHub repository URL is not inserted because repository access is not currently available in ChatGPT. Replace <YOUR_GITHUB_REPO_URL> with your real repository URL before submission.

Prepared for project cloning demonstration

1. What You Need Before Cloning

Before you go for the project submission, make sure the target computer has the required software. A fresh clone should not depend on files that exist only on your own laptop.

Requirement	Why it is needed
Git	Required to clone the repository. Check with: <code>git --version</code>
.NET SDK	Install the SDK version defined by the project .csproj (for example .NET 8). Check with: <code>dotnet --list-sdks</code>
SQL Server	Microsoft SQL Server / LocalDB / SQL Server Express, according to your project connection string.
Visual Studio 2022	Recommended for opening the solution and running ASP.NET Core. Install ASP.NET and web development workload.
SQL Server Management Studio (SSMS)	Recommended for restoring the database backup or running the SQL script.
Browser	Chrome / Edge for testing the application and Swagger/API pages.

Quick pre-check: On the submission computer, open Command Prompt and run the commands below. If both commands return version numbers, Git and .NET are ready.

```
git --version
dotnet --version
```

2. Recommended Submission Folder

Keep a USB backup as well as the GitHub repository. A safe submission folder can look like this:

```
VaultProject_Submission/
|-- Technical_Stack_Document.pdf
|-- Database/
|   |-- VaultDatabase.bak
|   |-- VaultDatabase.sql
|-- Test_Credentials.txt
`-- README.txt
```

Best practice: Bring both a .bak database backup and a .sql script if possible. If one restore method fails on the lab PC, you still have another option.

STEP 1 Copy the GitHub Repository URL

Open your final GitHub repository in the browser. Click Code, choose HTTPS, and copy the repository URL. It normally looks similar to:

```
https://github.com/<username>/<vault-repository>.git
```

Make sure: the final code is already merged into the main branch before cloning.

STEP 2 Open Command Prompt in the Destination Folder

For a clean demonstration, clone into a new empty folder. Example:

```
cd C:\Users\<YourName>\Desktop
mkdir VaultCloneTest
cd VaultCloneTest
```

STEP 3 Clone the Project

Run the following command using your real repository URL:

```
git clone <YOUR_GITHUB_REPO_URL>
```

Example only:

```
git clone https://github.com/your-user/personal-digital-vault.git
```

After cloning, enter the project folder:

```
cd <repository-folder-name>
```

Verify the clone: Run `dir` and confirm that the solution/project files, source folders, README, and other required files are present.

```
dir
```

Optional Git Check

```
git status
git branch
```

Expected result: the working tree should be clean, and the current branch should normally be main.

STEP 4 Restore NuGet Packages

If the repository contains a `.sln` file, run the restore command from the folder that contains the solution:

```
dotnet restore
```

A successful restore downloads the NuGet packages required by Entity Framework Core, ASP.NET Core authentication, Swagger, SQL Server provider, and other project dependencies.

STEP 5 Build the Project

```
dotnet build
```

Expected result: Build succeeded. If the build fails, do not continue to the demo until the error is fixed.

3. Database Setup on the Fresh Computer

The cloned source code does not automatically include your local SQL Server database. You must restore or create the database and then make sure the application connection string points to it.

Method A - Restore the .bak Backup (Recommended)

- Open SQL Server Management Studio (SSMS) and connect to the available SQL Server instance.
- Right-click Databases -> Restore Database.
- Choose Device -> Add -> select VaultDatabase.bak from your USB/submission folder.
- Choose the destination database name used by the project.
- Click OK and wait for the restore to complete.
- Expand Databases and confirm the tables and sample data are visible.

Method B - Run a .sql Script

If you bring a complete SQL script, open it in SSMS and execute it. The script should create the database/tables and insert any sample records needed for the demo.

Method C - Entity Framework Core Migrations (If Included)

If the repository already contains valid EF Core migrations and the connection string is correct, you can use:

```
dotnet ef database update
```

Note: This command requires the Entity Framework command-line tools and valid migrations. For a submission demo, a tested .bak or .sql file is usually safer.

Check appsettings.json

Open the backend appsettings.json file and check the ConnectionStrings section. Example formats are shown below; use the one that matches your actual SQL Server setup.

```
"ConnectionStrings": {
  "DefaultConnection": "Server=(localdb)\
\MSSQLLocalDB;Database=PersonalDigitalVaultDb;Trusted_Connection=True;TrustServerCertificate=True;"
}
```

If the lab PC uses SQL Server Express instead of LocalDB, the server name may be different, for example:

```
Server=.\
\SQLEXPRESS;Database=PersonalDigitalVaultDb;Trusted_Connection=True;TrustServerCertificate
=True;
```

Do not guess the server name: Open SSMS first and use the exact SQL Server instance that successfully connects on that computer.

STEP 6 Run the ASP.NET Core Backend

Method 1 - Command Prompt: enter the folder that contains the API .csproj file, then run:

```
cd <path-to-api-project>
dotnet run
```

When the API starts, the terminal will show one or more local URLs, such as `http://localhost:5xxx` or `https://localhost:7xxx`. Use the URL printed on that computer.

Method 2 - Visual Studio 2022

- Open the .sln solution file.
- Wait until NuGet restore completes.
- Set the API/web project as Startup Project if needed.
- Press Ctrl+F5 (run without debugger) or F5 (run with debugger).
- Allow the development HTTPS certificate prompt if Windows asks.

4. Start the Frontend

Use the method that matches your repository structure.

Case 1 - Frontend is inside ASP.NET Core wwwroot

If your HTML/CSS/JavaScript files are inside wwwroot and the backend serves static files, normally you only need `dotnet run`. Open the application URL printed in the terminal.

Case 2 - Frontend is a separate Node/React/Vite project

Only use these commands if the repository contains a separate frontend folder with `package.json`:

```
cd <frontend-folder>
npm install
npm run dev
```

Important: Do not run npm commands if your frontend is plain HTML/CSS/JavaScript inside wwwroot and there is no `package.json`.

5. Verify the Project Before Showing the Lecturer

Test	What to verify	Done
Login / Register	User authentication works and the test account can sign in.	[]
Dashboard	Dashboard loads without API errors.	[]
Documents & Folders	Create/view folders and document records; download/upload if part of the build.	[]
Credentials & Categories	Credential/category pages load and database operations work.	[]
Profile / Admin	Profile and admin pages load for the correct role.	[]
Database	New data is actually saved and remains after refreshing.	[]
API	Swagger or API endpoints return successful responses if Swagger is enabled.	[]
Browser console	No critical JavaScript/API connection errors.	[]

6. Full Command Sequence - Quick Reference

This is the short command flow to remember during the cloning demonstration:

```
git clone <YOUR_GITHUB_REPO_URL>
cd <repository-folder>
dotnet restore
dotnet build
cd <api-project-folder>
dotnet run
```

If your project uses a separate Node frontend:

```
cd <frontend-folder>
npm install
npm run dev
```

7. Common Problems and Fast Fixes

Problem	Fix
git is not recognized	Git is not installed or not added to PATH. Install Git for Windows, close Command Prompt, and reopen it.
dotnet is not recognized	Install the required .NET SDK. Verify with <code>dotnet --version</code> .
Could not find a project to run	You are in the wrong folder. Use <code>dir</code> and <code>cd</code> into the folder that contains the <code>.csproj</code> file.
NuGet package restore error	Run <code>dotnet restore</code> . Check internet access and confirm project packages are compatible with the target .NET version.
SQL connection error / server not found	Open SSMS and confirm the server instance. Correct <code>appsettings.json</code> ConnectionStrings and ensure SQL Server is running.
Cannot open database requested by login	The database has not been restored/created, or the connection string database name is wrong.
EF migration command not found	Install <code>dotnet-ef</code> only if your demo actually needs migrations. Otherwise restore the prepared <code>.bak/.sql</code> dataset.
Frontend shows Failed to fetch / API error	Confirm the backend is running and the frontend API base URL uses the backend URL/port shown by <code>dotnet run</code> .
HTTPS certificate warning	For local development, trust the dev certificate with <code>dotnet dev-certs https --trust</code> if permitted on the computer.
Project works on your laptop but not after clone	A required file may be excluded by <code>.gitignore</code> , the database is local-only, or machine-specific paths/secrets are hard-coded. Fix this before submission.

8. Final Test to Do Today

Critical test: Do not test only in your existing development folder. Create a brand-new folder, clone the GitHub repository again, restore the database, and run the entire system exactly as you will do during submission.

[] Push/merge the final working code to main.

- [] Create an empty folder on your PC.
- [] Clone the repo from GitHub into that folder.
- [] Restore packages and build.
- [] Restore/create the SQL Server database from the dataset you will carry.
- [] Run backend and frontend.
- [] Test login and the main CRUD features.
- [] Write down a working test username/password.
- [] Copy database backup/script and documents to a USB drive.
- [] Keep an internet backup of the repo and PDF.

9. What to Say During the Demo

A simple explanation you can use:

"First I clone the final project from our GitHub main branch. Then I restore the .NET dependencies, configure/restore the SQL Server database dataset, build the solution, and run the ASP.NET Core API. Finally I open the frontend and verify authentication, database operations, and the main Vault modules."

Submission goal: The lecturer should be able to see that the repository is complete, the project can be cloned on a fresh machine, the required dataset/database is available, and the system runs without relying on your old local project folder.

END OF GUIDE